arXiv:2112.05682v3 [cs.LG] 10 Oct 2022

SELF-ATTENTION DOES NOT NEED $O(n^2)$ MEMORY

A PREPRINT

Markus N. Rabe and Charles Staats
Google Research
{mrabe,cstaats}@google.com

ABSTRACT

We present a very simple algorithm for attention that requires $O(1)$ memory with respect to sequence length and an extension to self-attention that requires $O(\log n)$ memory. This is in contrast with the frequently stated belief that self-attention requires $O(n^2)$ memory. While the time complexity is still $O(n^2)$, device memory rather than compute capability is often the limiting factor on modern accelerators. Thus, reducing the memory requirements of attention allows processing of longer sequences than might otherwise be feasible. We provide a practical implementation for accelerators that requires $O(\sqrt{n})$ memory, is numerically stable, and is within a few percent of the runtime of the standard implementation of attention. We also demonstrate how to differentiate the function while remaining memory-efficient. For sequence length 16384, the memory overhead of self-attention is reduced by 59X for inference and by 32X for differentiation.

1 Introduction

Attention (Bahdanau et al., 2015) is widely used in modern neural architectures. In particular, it is the heart of the Transformer architecture (Vaswani et al., 2017), which has revolutionized Natural Language Processing (Devlin et al., 2019), and found wide-spread adoption across several research areas since then.

Given a query $q \in \mathbb{R}^d$ and lists of keys and values $k_1, \ldots, k_n$ and $v_1, \ldots, v_n \in \mathbb{R}^d$ of length n, *attention* is defined as follows:

$$s_i = \mathrm{dot}(q, k_i), \qquad s'_i = \frac{e^{s_i}}{\sum_j e^{s_j}}, \qquad \mathrm{attention}(q, k, v) = \sum_i v_i s'_i.$$

The result of the attention operation for a single query, is hence a weighted sum of the value vectors, where the weights are the softmax of the dot products of the query and the keys.

The straight-forward implementation of the attention operation above requires us to first compute and remember s_i for all i, leading to a $O(n)$ time and memory complexity for each query. Transformers use *self-attention*, which issues a separate query for each position in the sequence, so the overall time and space complexity is $O(n^2)$.

In many works the quadratic time and space complexity of self-attention has been used as the motivation for the investigation of variants of the original attention mechanism and architectures with more favorable complexity classes (Kitaev et al., 2020; Roy et al., 2021; Zaheer et al., 2020; Choromanski et al., 2020; Wang et al., 2020; Ren et al., 2021; Child et al., 2019; Tay et al., 2021; Wang et al., 2020; Ma et al., 2021; Shen et al., 2021; Qiu et al., 2020). Modern accelerator hardware, such as GPUs and TPUs, are often memory constrained for applications in deep learning, while compute is relatively cheap. So the space complexity of transformers is a particular concern, c.f. Kitaev et al. (2020); Roy et al. (2021); Zaheer et al. (2020).

In this work, we present new algorithms for attention and self-attention that require only constant memory and logarithmic memory, respectively. The basic algorithm is very simple; but it requires a trick to make it numerically feasible (see Section 3). We also present an implementation in JAX (Bradbury et al., 2018), which runs efficiently on TPUs, and requires $O(\sqrt{n})$ memory for self-attention (see Section 4).

Unlike other works that aim to reduce the memory complexity of attention, the memory-efficient algorithm for attention that we suggest is not an approximation, but computes the same function. We can hence use the memory-efficient

arXiv:2112.05682v3 [cs.LG] 10 Oct 2022

自注意力机制无需 $O(n^2)$ 量级的记忆空间

一份预印本

Markus N. Rabe与Charles Staats，谷歌研究院
{mrabe,cstaats}@google.com

摘要

我们提出了一种极为简单的注意力机制算法，其针对序列长度的 $O(1)$内存需求；同时还给出了该机制在自注意力机制上的扩展版本，其内存需求仅为 $O(\log n)$。这一结果与人们普遍认为自注意力机制需要 $O(n^2)$内存的观点截然不同。尽管该算法的时间复杂度仍为 $O(n^2)$，但在现代加速器上，限制性能发挥的因素往往是设备内存而非计算能力。因此，降低注意力机制的内存需求使得处理更长的序列成为可能。我们为加速器实现了一种实用方案，它仅需 $O(\sqrt{n})$内存，数值稳定性优异，且运行时间仅比标准实现快几个百分点。此外，我们还展示了如何在保持内存高效利用的同时对相关函数进行求导。在序列长度为16384的情况下，自注意力机制在推理过程中的内存开销可减少59倍，而在求导过程中的内存开销则可减少32倍。

1 引言

注意力机制（Bahdanau等人，2015年）在现代神经网络架构中被广泛运用。尤其是，它构成了Transformer架构（Vaswani等人，2017年）的核心，而该架构彻底改变了自然语言处理领域（Devlin等人，2019年），此后也在众多研究领域得到了广泛的应用。

给定一个查询 $q \in \mathbb{R}^d$ 以及长度为 n的键值列表 k_1、……、 k_n 和 v_1、……、 $v_n \in \mathbb{R}^d$，注意力机制的定义如下：

$$s_i = \mathrm{dot}(q, k_i), \qquad s'_i = \frac{e^{s_i}}{\sum_j e^{s_j}}, \qquad \mathrm{attention}(q, k, v) = \sum_i v_i s'_i.$$

因此，对于单个查询而言，注意力运算的结果就是各价值向量的加权之和，而这些权重则是该查询与键值对进行点积后经过softmax函数处理得到的结果。

上述注意力运算的直接实现方式要求我们首先为所有的 i计算并存储 s_i，从而导致每次查询都需要 $O(n)$的时间与内存复杂度。而Transformer模型则采用自注意力机制，它会为序列中的每个位置分别生成一个查询，因此整体的时间与空间复杂度仅为 $O(n^2)$。

在许多研究中，自注意力机制的二次时间与空间复杂度被用作探索该机制变体以及具有更优复杂度特性的架构的动力（Kitaev等人（2020年）；Roy等人（2021年）；Zaheer等人（2020年）；Choromanski等人（2020年）；Wang等人（2020年）；Ren等人（2021年）；Child等人（2019年）；Tay等人（2021年）；Wang等人（2020年）；Ma等人（2021年）；Shen等人（2021年）；Qiu等人（2020年））。在深度学习应用中，诸如图形处理器和TPU之类的现代加速器硬件往往存在内存限制，而计算资源相对较为充足。因此，Transformer模型的空间复杂度尤其值得关注，参见Kitaev等人（2020年）；Roy等人（2021年）；Zaheer等人（2020年）的相关论述。

在本研究中，我们提出了新的注意力机制与自注意力机制算法，前者仅需常数级内存，后者则仅需对数级内存。这类基本算法本身十分简单，但为使其在数值计算上可行，仍需采用一些特殊技巧（详见第3节）。此外，我们还展示了基于JAX框架的实现方案（Bradbury等人，2018年），该方案能在TPU芯片上高效运行，且自注意力机制部分仅需 $O(\sqrt{n})$的内存资源（详见第4节）。

与那些旨在降低注意力机制内存复杂度的其他研究不同，我们提出的这种高内存效率算法并非近似方案，而是能够计算出完全相同的函数。因此，我们可以采用这种高内存效率的

algorithm as a drop-in replacement for other attention implementations to save memory. This may allow us to re-consider architecture choices, or scale to new datasets that require longer, dense attention. However, our algorithm still requires $O(n^2)$ time complexity for self-attention and $O(n)$ time complexity for single-query attention, and the various efficient, long-context attention mechanisms remain an interesting alternative to (dense) attention.

2 Algorithm

First, we present the algorithm for the attention operation with a single query and extend the algorithm to self-attention at the end of this Section. We observe that the division by $\sum_j e^{s_j}$ can be moved to the very end of the attention operation using the distributive law:

$$s_i = \mathrm{dot}(q, k_i), \qquad s'_i = e^{s_i}, \qquad \mathrm{attention}(q, k, v) = \frac{\sum_i v_i s'_i}{\sum_j s'_j}. \tag{1}$$

After publishing our initial draft, we were made aware that (1) is a rediscovery of the "lazy softmax" method of Jang et al. (2019, equation 4). Unfortunately their paper went in a different direction and did not discuss the memory complexity implications and other innovations we present in the remainder of this paper. For more details see Section 6.

This can be computed with constant memory: The memory overhead of this algorithm consists of a vector $v^* \in \mathbb{R}^d$ and a scalar $s^* \in \mathbb{R}$, both initialized with 0. Given the query q, keys $k_1, \ldots, k_n$ and values $v_1, \ldots, v_n$, we process the keys and values in sequence. Given a key value pair k_i, v_i, we compute $s_i = \mathrm{dot}(q, k_i)$ and update $v^* \leftarrow v^* + v_i e^{s_i}$ and $s^* \leftarrow s^* + e^{s_i}$. After processing all keys and values, we divide $\frac{v^*}{s^*}$ to get the final result.

The analysis of space complexity assumes that inputs are given in a particular order: we first read the query, and then a list of *pairs* of keys and values. If the inputs are provided in a different order, we have to additionally store an index into the sequence, requiring $O(\log n)$ memory instead.

To extend this algorithm to *self*-attention, we compute the results to all queries sequentially. This requires just one additional index into the list of queries, giving rise to the $O(\log n)$ memory complexity. Note that the operation produces outputs that are linear in the size of the number of queries, i.e., $O(n)$, which is not counted towards the space complexity.

3 Numerical Stability

The formulation of standard attention that we presented in the Introduction, as well as our memory-efficient algorithm, are not numerically stable when using floating point arithmetic, because the softmax exponentiates the scores. For scores ≥ 89 the exponentiation results in `inf` (for `bfloat16` and `float32`), which will be carried through to the final result of the attention operation. In practice, the softmax is implemented by subtracting the maximum score from all scores. This does not change the result of the softmax, but avoids this numerical problem.

Our incremental computation of the sum of exponentiated scores (and the values times the scores) does not immediately allow for the same trick, as the maximum may depend on the last score in the sequence. But the subtraction cannot be delayed either, since the scores must be exponentiated before they can be added to the cumulative sum.

To resolve this problem, we introduce an additional scalar, which keeps track of the maximum score that the incremental algorithm has seen so far, and we renormalize the sums of exponentiated values as needed: We initialize the vector $v^* \in \mathbb{R}^d$ and scalar $s^* \in \mathbb{R}$ with 0, and m^* with $-\inf$. As before, given a key value pair k_i, v_i, we compute $s_i = \mathrm{dot}(q, k_i)$, but then the algorithm differs slightly from Section 2. We first compute $m_i = \max(m^*, s_i)$ and update $v^* \leftarrow v^* e^{m^* - m_i} + v_i e^{s_i - m_i}$ and $s^* \leftarrow s^* e^{m^* - m_i} + e^{s_i - m_i}$ and $m^* \leftarrow m_i$. After processing all keys and queries, we divide $\frac{v^*}{s^*}$ to get the final result.

4 An Implementation For TPUs

In this section, we provide a version of the algorithm above that exploits the massive parallelism of modern hardware, such as GPUs or TPUs. The naive algorithm above is is not trivial to parallelize for a compiler, as the incremental sum introduces a dependency across all keys and values.

We present the entire implementation, including the support for multiple attention heads and memory-efficient differentiation in Figure 1. The implementation does not optimize strictly for memory efficiency, but instead aims to strike a balance between simplicity, computational efficiency, and memory requirements.

该算法可作为其他注意力机制的直接替代方案，从而节省内存。这或许让我们有机会重新审视架构设计，或应用于那些需要更长且密集的注意力机制的新数据集。不过，我们的算法在处理自注意力机制时仍需 $O(n^2)$的时间复杂度，而在处理单查询注意力机制时则需要 $O(n)$的时间复杂度；与此同时，各种高效的长上下文注意力机制依然是(密集)注意力机制的一种有趣替代选择。

2 算法

首先，我们将介绍具有单个查询的注意力运算算法，并在本节末尾将该算法扩展为自注意力机制。我们发现，利用分配律，除以 $\sum_j e^{s_j}$ 的操作可以被移到注意力运算的最后一步。

$$s_i = \mathrm{dot}(q, k_i), \qquad s'_i = e^{s_i}, \qquad \mathrm{attention}(q, k, v) = \frac{\sum_i v_i s'_i}{\sum_j s'_j}. \tag{1}$$

在发布初步草稿后，我们发现：(1)实际上是对Jang等人（2019年，公式4）提出的“懒惰softmax算法”的重新发现。遗憾的是，他们的论文探讨的方向有所不同，既没有涉及内存复杂度方面的问题，也没有提及我们在本文其余部分提出的其他创新点。更多细节请参见第6节。

该运算可在常量级内存消耗下完成：该算法的内存开销仅包括一个被初始化为0的向量 $v^* \in \mathbb{R}^d$和一个被初始化为0的标量 $s^* \in \mathbb{R}$。给定查询 q、键 k_1、……、 k_n 以及值 v_1、……、 v_n后，我们会按顺序处理这些键值对。对于每一组键值对 k_i、 v_i，我们都会计算 $s_i =$ 点积(q、 k_i)，并据此更新 $v^* \leftarrow v^* + v_i e^{s_i}$和 $s^* \leftarrow s^* + e^{s_i}$的值。在处理完所有键值对之后，只需将$\frac{v^*}{s^*}$进行除法运算即可得到最终结果。

对空间复杂度的分析假定输入是按特定顺序给出的：我们首先读取查询，然后再读取键值对列表。如果输入的顺序不同，我们就必须额外在序列中存储索引，这样就需要 $O(\log n)$的内存空间。

要将该算法扩展到自注意力机制中，我们需要依次为所有查询计算结果。这仅需在查询列表中多一个索引即可实现，因此其内存复杂度为 $O(\log n)$。需要注意的是，该运算产生的输出量与查询数量成线性关系，即 $O(n)$，这一数值并不计入空间复杂度之中。

3 数值稳定性

在引言中介绍的标准注意力机制及其对应的高内存效率算法，在使用浮点运算时并不具备数值稳定性，原因是softmax函数会对各项得分进行指数运算。对于得分 ≥ 89 而言，这种指数运算会在bfloat16和float32数据类型下产生无穷大值，而这些异常值会影响到注意力运算的最终结果。实际上，softmax函数通常是通过从所有得分中减去最高得分来实现的。虽然这种方式不会改变softmax函数的计算结果，却能有效避免上述数值问题。

我们对经过指数运算的得分（以及这些得分与对应数值的乘积）所进行的逐步累积计算，并无法立即使用同样的技巧，因为最大值可能取决于序列中的最后一个得分。不过减法操作也同样不能推迟，因为在将这些得分加入累积和之前，必须先对它们进行指数运算。

为了解决这个问题，我们引入了一个额外的标量，用于记录当前算法迄今为止观测到的最高得分，并根据需要对经过指数运算后的值之和进行重新归一化处理：我们将向量 $v^* \in \mathbb{R}^d$ 和标量 $s^* \in \mathbb{R}$ 初始化为0，而 m^* 和 –则初始化为无穷大。与之前相同，给定一个键值对 k_i, v_i后，我们会计算s_i =等于q与 k_i的点积，但此处的算法与第2节略有不同。首先计算 m_i =等于m^*与 s_i中的较大值，随后更新 $v^* \leftarrow v^* e^{m^* - m_i} + v_i e^{s_i - m_i}$ 、 $s^* \leftarrow s^* e^{m^* - m_i} + e^{s_i - m_i}$ 和 $m^* \leftarrow m_i$。在处理完所有的键和查询之后，再通过将$\frac{v^*}{s^*}$除以某个值来得到最终结果。

4 张量处理单元的实现方案

在本节中，我们给出了上述算法的一个改进版本，该版本充分利用了GPU或TPU等现代硬件所具备的强大并行处理能力。需要注意的是，原有的朴素算法并不容易被编译器进行并行化处理，因为累加操作会在所有的键与值之间产生依赖关系。

我们在图1中展示了完整的实现方案，包括对多个注意力头的支持以及内存效率较高的微分计算方式。该实现并非专门为极致的内存效率而优化，而是力求在简洁性、计算效率与内存需求之间取得平衡。

```python
import functools, jax, math
from jax import numpy as jnp

def _query_chunk_attention(query, key, value, precision, key_chunk_size=4096):
  """Multi-head dot product attention with a limited number of queries."""
  num_kv, num_heads, k_features = key.shape
  v_features = value.shape[-1]
  key_chunk_size = min(key_chunk_size, num_kv)
  query = query / jnp.sqrt(k_features)

  @functools.partial(jax.checkpoint, prevent_cse=False)
  def summarize_chunk(query, key, value):
    attn_weights = jnp.einsum('qhd,khd->qhk', query, key, precision=precision)
    max_score = jnp.max(attn_weights, axis=-1, keepdims=True)
    max_score = jax.lax.stop_gradient(max_score)
    exp_weights = jnp.exp(attn_weights - max_score)
    exp_values = jnp.einsum('vhf,qhv->qhf', value, exp_weights, precision=precision)
    return (exp_values, exp_weights.sum(axis=-1),
            max_score.reshape((query.shape[0], num_heads)))

  def chunk_scanner(chunk_idx):
    key_chunk = jax.lax.dynamic_slice(
        key, (chunk_idx, 0, 0),
        slice_sizes=(key_chunk_size, num_heads, k_features))
    value_chunk = jax.lax.dynamic_slice(
        value, (chunk_idx, 0, 0),
        slice_sizes=(key_chunk_size, num_heads, v_features))
    return summarize_chunk(query, key_chunk, value_chunk)

  chunk_values, chunk_weights, chunk_max = jax.lax.map(
      chunk_scanner, xs=jnp.arange(0, num_kv, key_chunk_size))

  global_max = jnp.max(chunk_max, axis=0, keepdims=True)
  max_diffs = jnp.exp(chunk_max - global_max)
  chunk_values *= jnp.expand_dims(max_diffs, axis=-1)
  chunk_weights *= max_diffs

  all_values = chunk_values.sum(axis=0)
  all_weights = jnp.expand_dims(chunk_weights, -1).sum(axis=0)
  return all_values / all_weights

def attention(query, key, value, precision=jax.lax.Precision.HIGHEST,
              query_chunk_size=1024):
  """Memory-efficient multi-head dot product attention."""
  num_q, num_heads, q_features = query.shape

  def chunk_scanner(chunk_idx, _):
    query_chunk = lax.dynamic_slice(
        query, (chunk_idx, 0, 0),
        slice_sizes=(min(query_chunk_size, num_q), num_heads, q_features))
    return (chunk_idx + query_chunk_size,
            _query_chunk_attention(query_chunk, key, value, precision=precision))

  _, res = jax.lax.scan(
      chunk_scanner, init=0, xs=None, length=math.ceil(num_q / query_chunk_size))
  return res.reshape(num_q, num_heads, value.shape[-1])
```

Figure 1: Implementation of memory-efficient attention suited for TPUs.


```python
import functools, jax, math
from jax import numpy as jnp

def _query_chunk_attention(query, key, value, precision, key_chunk_size=4096):
  """Multi-head dot product attention with a limited number of queries."""
  num_kv, num_heads, k_features = key.shape
  v_features = value.shape[-1]
  key_chunk_size = min(key_chunk_size, num_kv)
  query = query / jnp.sqrt(k_features)

  @functools.partial(jax.checkpoint, prevent_cse=False)
  def summarize_chunk(query, key, value):
    attn_weights = jnp.einsum('qhd,khd->qhk', query, key, precision=precision)
    max_score = jnp.max(attn_weights, axis=-1, keepdims=True)
    max_score = jax.lax.stop_gradient(max_score)
    exp_weights = jnp.exp(attn_weights - max_score)
    exp_values = jnp.einsum('vhf,qhv->qhf', value, exp_weights, precision=precision)
    return (exp_values, exp_weights.sum(axis=-1),
            max_score.reshape((query.shape[0], num_heads)))

  def chunk_scanner(chunk_idx):
    key_chunk = jax.lax.dynamic_slice(
        key, (chunk_idx, 0, 0),
        slice_sizes=(key_chunk_size, num_heads, k_features))
    value_chunk = jax.lax.dynamic_slice(
        value, (chunk_idx, 0, 0),
        slice_sizes=(key_chunk_size, num_heads, v_features))
    return summarize_chunk(query, key_chunk, value_chunk)

  chunk_values, chunk_weights, chunk_max = jax.lax.map(
      chunk_scanner, xs=jnp.arange(0, num_kv, key_chunk_size))

  global_max = jnp.max(chunk_max, axis=0, keepdims=True)
  max_diffs = jnp.exp(chunk_max - global_max)
  chunk_values *= jnp.expand_dims(max_diffs, axis=-1)
  chunk_weights *= max_diffs

  all_values = chunk_values.sum(axis=0)
  all_weights = jnp.expand_dims(chunk_weights, -1).sum(axis=0)
  return all_values / all_weights

def attention(query, key, value, precision=jax.lax.Precision.HIGHEST,
              query_chunk_size=1024):
  """Memory-efficient multi-head dot product attention."""
  num_q, num_heads, q_features = query.shape

  def chunk_scanner(chunk_idx, _):
    query_chunk = lax.dynamic_slice(
        query, (chunk_idx, 0, 0),
        slice_sizes=(min(query_chunk_size, num_q), num_heads, q_features))
    return (chunk_idx + query_chunk_size,
            _query_chunk_attention(query_chunk, key, value, precision=precision))

  _, res = jax.lax.scan(
      chunk_scanner, init=0, xs=None, length=math.ceil(num_q / query_chunk_size))
  return res.reshape(num_q, num_heads, value.shape[-1])
```

图1：专为张量处理单元设计的、具备高内存效率注意力机制的实现方案。

Sequence length	$n = 2^8$	2^{10}	2^{12}	2^{14}	2^{16}	2^{18}	2^{20}
Size of inputs and outputs	160KB	640KB	2.5MB	10MB	40MB	160MB	640MB
Memory overhead of standard attention	270KB	4.0MB	64MB	1GB	OOM	OOM	OOM
Memory overhead of memory-eff. attn.	270KB	4.0MB	16MB	17MB	21MB	64MB	256MB
Compute time on TPUv3	0.06ms	0.11ms	0.7ms	11.3ms	177ms	2.82s	45.2s
Relative compute speed	±5%	±5%	-8±2%	-13±2%	-	-	-

Table 2: Memory and time requirements of self-attention during **inference**.

To exploit the parallelism available in modern hardware, we split the computation into chunks at the cost of some additional memory. In the outer loop (lines 54-55), we split the queries in to chunks of constant size, resulting in a linear number of iterations. In each iteration of the outer loop, we call `_query_chunk_attention`, which itself processes the keys and values in chunks (lines 30-31). The chunks are processed sequentially and each chunk is summarized independently (lines 12 to 19). Assuming a chunk size of $\sqrt{n}$ for the keys and values, we hence obtain $\sqrt{n}$ summaries, giving rise to the $O(\sqrt{n})$ memory complexity.

After the summaries are computed, they need to be rescaled (lines 33 to 36) along the lines of Section 3, before we return the values divided by the weights (line 40). The result of each iteration of the outer loop is directly written to the output tensor `res` (line 54), so that no additional memory is consumed across iterations. (A multi-stage summarization approach could achieve $O(\log n)$ but would complicate the implementation.)

While a constant chunk size for the queries and a chunk size of $\sqrt{n}$ for the keys and values is optimal for memory consumption, the runtime is also affected by the choice of chunk size in practice, which is heavily affected by the choice of hardware. Ultimately, we have to leave this trade-off to the programmer, and expose the chunk sizes as arguments `query_chunk_size` and `key_chunk_size`. In Figure 1 we provide default values for the chunk sizes that lead to minimal runtime impact on TPU, while still providing significant memory savings.

5 Empirical Analysis

In this section, we experimentally compare the memory requirements and runtime performance of the suggested algorithm compared to the implementation of attention currently provided by Flax (Heek et al. (2020), see `flax/linen/attention.py`). We open-sourced the code of our implementation and most of the evaluation as a colab to help others reproduce the results: `https://github.com/google-research/google-research/tree/master/memory_efficient_attention`.

5.1 Inference

In Table 2 we compare the memory requirements and the compute time of the memory-efficient attention implementation and the Flax implementation of attention. The size of inputs and outputs includes the query, key, and value tensors of dtype `bfloat16`, and the output tensor of dtype `float32`. We measure the memory overhead as the TPUs peak memory in excess of the input and output tensors. All computations were done on a single TPUv3 chip. For this experiment, we only use one attention head.

Our memory-efficient implementation of attention removes the memory bottleneck of self-attention, scaling at least to a sequence length of 1M. At this sequence length the algorithm is multiplying over 1 trillion combinations of queries and keys. The time complexity is still quadratic.

The "relative compute speed" of the implementations was computed as the median over 100 runs—but the numbers still fluctuated across multiple runs of the evaluation and we only provide them to demonstrate that the runtime performance is roughly similar. Please note that this experiment analyzes the attention operation in isolation; the measured relative performance is not necessarily the same when the operations are embedded in larger architectures. In fact, we observed a slight increase in steps/sec of about 4% when training a small Transformer.

For all cases where the standard attention would not OOM (i.e. require $>$ 16GB device memory), we checked that the results of the two implementations are within 1.8×10^{-7} for inputs drawn from a normal distribution with standard deviation 1 (measured as the maximal absolute difference of any dimension in a self-attention over sequence length 2^{14}).

序列长度	$n = 2^8$	2^{10}	2^{12}	2^{14}	2^{16}	2^{18}	2^{20}
输入与输出的数据规模	160KB	640KB	2.5MB	10MB	40MB	160MB	640MB
标准注意力机制的内存开销	270KB	4.0MB	64MB	1GB	OOM	OOM	OOM
高效注意力机制所带来的内存开销。	270KB	4.0MB	16MB	17MB	21MB	64MB	256MB
在TPUv3上的计算时间。	0.06ms	0.11毫秒	0.7毫秒	11.3毫秒	177毫秒	2.82秒	45.2秒
相对计算速度	±5%	±5%	-8±2%	-13±2%	-	-	-

表2：推理过程中自注意力机制的内存与时间需求。

为充分利用现代硬件中的并行处理能力，我们以消耗少量额外内存为代价将计算任务拆分为多个块。在外层循环（第54-55行）中，我们会将查询项划分为大小恒定的块，从而使迭代次数呈线性增长。在外层循环的每次迭代中，都会调用_query_chunk_attention函数，该函数会进一步按块处理键值数据（第30-31行）。这些块会按顺序被处理，且每个块都会被独立汇总（第12至19行）。假设键值数据的块大小为$\sqrt{n}$，那么最终就能得到$\sqrt{n}$ 个汇总结果，由此便产生了 $O(\sqrt{n})$级别的内存复杂度。
在计算出摘要之后，需要按照第3节的方法对它们进行重新缩放（第33至36行），随后再将这些值除以权重并返回（第40行）。外层循环的每次迭代结果都会直接写入输出张量res中（第54行），这样就不会在各次迭代之间占用额外的内存。（虽然采用多阶段总结方法可以达到 $O(\log n)$的效果，但会使得实现变得更加复杂。）

虽然对查询使用固定大小的块，而对键和值使用 $\sqrt{n}$ 大小的块是控制内存消耗的最佳方案，但实际上运行时间也会受到块大小选择的影响，而这一选择又极大程度上取决于硬件的性能。最终，我们不得不将这种权衡交给程序员处理，并通过query_chunk_size和key_chunk_size这两个参数来指定不同的块大小。在图1中，我们给出了这些块大小的默认值，它们能够在为TPU带来最小运行时间影响的同时，依然实现显著的内存节省。

5 实证分析

在本节中，我们通过实验对比了所提出的算法与Flax框架当前实现的注意力机制在内存需求及运行时性能方面的差异（参考Heek等人（2020年）的成果，见flax/linen/attention.py）。为便于他人复现实验结果，我们已将实现代码及大部分评估内容以Colab的形式开源，地址如下：
https://github.com/google-research/google-research/tree/master/memory_efficient_attention。

5.1 推理过程

在表2中，我们对比了高内存效率注意力机制实现与Flax框架中注意力机制实现所需的内存容量及计算时间。输入和输出的数据规模包括类型为bfloat16的查询张量、键张量和值张量，以及类型为float32的输出张量。所谓内存开销，指的是TPU芯片的峰值内存使用量超出输入和输出张量所占内存后的额外部分。所有计算均在单块TPUv3芯片上完成。在本实验中，我们仅使用一个注意力头。
我们采用的低内存消耗的注意力机制实现消除了自注意力机制的内存瓶颈，其处理能力至少可支撑100万级别的序列长度。在如此长的序列长度下，该算法仍需对超过1万亿组查询项与键项进行乘法运算，其时间复杂度依然为二次方级。

这些实现的“相对计算速度”是通过100次运行后的中位数来计算的——但即便在多次评估运行中，相关数值依然存在波动，我们给出这些数据只是为了说明其运行时性能大致相当。需注意的是，该实验是单独分析注意力运算的；当这些运算被嵌入到更复杂的架构中时，所测得的相对性能未必保持一致。实际上，在训练小型Transformer模型时，我们观察到了每秒处理步数约有4%的轻微上升。
在标准注意力机制不会引发内存溢出的所有情况下（即无需 $>$ 16GB的设备内存），我们对从标准差为1的正态分布中抽取的输入进行了测试，结果表明两种实现方法的结果差异仍在 1.8×10^{-7} 范围内——该差异是通过计算序列长度为 2^{14}时的自注意力机制在各维度上的最大绝对差值来衡量的。

Sequence length	$n = 2^8$	2^{10}	2^{12}	2^{14}	2^{16}	2^{18}	2^{20}
Size of inputs and outputs	192KB	768KB	2.9MB	12MB	47MB	188MB	750MB
Memory overhead of standard attention	532KB	8.0MB	128MB	2.0GB	OOM	OOM	OOM
Memory overhead of memory-eff. attn.	532KB	8.0MB	41MB	64MB	257MB	1.0GB	4.0GB
Compute time on TPUv3	0.1ms	0.18ms	1.4ms	21ms	336ms	5.3s	85s
Relative compute speed	±5%	±5%	-30±5%	-35±5%	-	-	-

Table 3: Memory and time requirements of self-attention during **differentiation**. Note that the slowdown in compute speed is expected due to the use of checkpointing in memory-efficient attention.

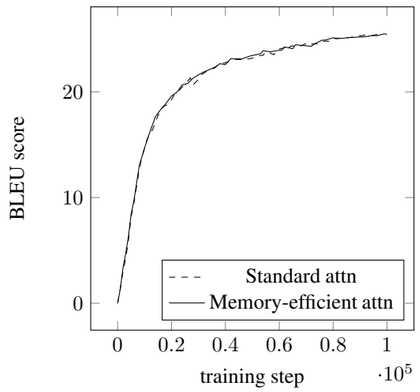


Figure 4: BLEU scores of a two Transformer models trained with standard attention and memory-efficient attention.

5.2 Differentiation

During the forward pass our algorithm saves memory by summarizing parts of the attention matrix sequentially, allowing it to forget the parts of the attention matrix it has summarized already. A naive application of differentiation would have to store all those intermediate results and our algorithm would loose its memory advantage entirely. So we apply checkpointing (Chen et al., 2016) in line 11 to the function that summarizes the individual chunks. The intermediate results can thus be forgotten during the forward pass and recomputed during backpropagation.

In Table 3 we compare runtime and peak memory during differentiation of our implementation to standard attention. We used the same setting as for the forward pass, but applied `jax.grad` to an arbitrarily chosen loss function (the sum of the results). The relative compute speed was reduced significantly compared to standard attention. This is expected when using checkpointing since some values must be recomputed during backpropagation.

Note that applying checkpointing to the standard attention algorithm would not achieve these results. The standard algorithm with checkpointing would forget the attention matrix after it is formed; our algorithm never forms the full attention matrix at all.

5.3 Training

We integrated our memory-efficient implementation into a simple Transformer architecture provided in the Flax library, and ran the WMT en-de translation experiment with the standard attention module and with the memory-efficient attention module. Throughout the training, the two implementations behaved almost identically. After 100K training steps, the evaluation accuracy reached 62.69 for the memory-efficient implementation and 62.59 for the standard implementation. This demonstrates that our memory-efficient implementation of self-attention can be used to replace existing implementations. Figure 4 illustrates that both models resulted in very similar BLEU scores. We used the default settings for the WMT en-de experiment as given in the Flax implementation, except that we had to deactivate example packing to simplify the masking code. This also required us to lower the learning rate to 0.005.

序列长度	$n = 2^8$	2^{10}	2^{12}	2^{14}	2^{16}	2^{18}	2^{20}
输入与输出的数据规模	192KB	768KB	2.9MB	12MB	47MB	188MB	750MB
标准注意力机制的内存开销	532KB	8.0MB	128MB	2.0GB	OOM	OOM	OOM
高内存效率的注意力机制所产生的内存开销	532KB	8.0MB	41MB	64MB	257MB	1.0GB	4.0GB
在TPUv3上的计算时间	0.1ms	0.18ms	1.4ms	21ms	336毫秒	5.3s	85s
相对计算速度	±5%	±5%	-30±5%	-35±5%	-	-	-

表3：在求导过程中自注意力机制所需的内存与时间资源。需要注意的是，由于采用了内存高效注意力机制中的检查点机制，计算速度会出现一定程度的下降。

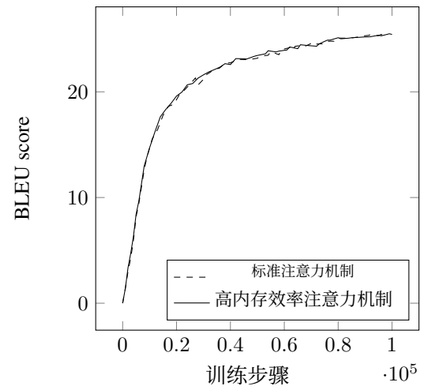


图4：BLEU分值 即使用标准注意力机制和高内存效率注意力机制训练的两个Transformer模型的BLEU分数差异。

5.2 微分计算

在前向传播过程中，我们的算法通过逐次对注意力矩阵的某些部分进行汇总来节省内存，这样它就可以遗忘已经处理过的那些部分。如果直接采用常规的微分方法，就必须存储所有中间结果，这样一来我们的算法就完全丧失了内存优势。因此我们在第11行针对负责汇总各个数据块的函数采用了检查点机制（Chen等人，2016年）。如此一来，这些中间结果便可以在前向传播时被遗忘，而在反向传播时再重新计算。

在表3中，我们对比了在本实现中进行微分计算时的运行时间以及峰值内存占用情况与采用标准注意力机制时的表现。我们使用了与前向传播相同的配置，但将jax.gradto应用于一个任意选取的损失函数（即各结果之和）。与标准注意力机制相比，其相对计算速度有了显著下降。由于在反向传播过程中需要重新计算某些数值，出现这种现象是意料之中的。

需要特别注意的是，即便对标准注意力算法应用检查点机制，也无法得到这样的结果。采用检查点机制的标准算法会在注意力矩阵生成后被遗忘；而我们的算法从一开始就不会生成完整的注意力矩阵。

5.3 训练

We integrated our memory-efficient implementation into a simple Transformer architecture provided in the Flax library, and ran the WMT en-de translation experiment with the standard attention module and with the memory-efficient attention module. Throughout the training, the two implementations behaved almost identically. After 100K training steps, the evaluation accuracy reached 62.69 for the memory-efficient implementation and 62.59 for the standard implementation. This demonstrates that our memory-efficient implementation of self-attention can be used to replace existing implementations. Figure 4 illustrates that both models resulted in very similar BLEU scores. We used the default settings for the WMT en-de experiment as given in the Flax implementation, except that we had to deactivate example packing to simplify the masking code. This also required us to lower the learning rate to 0.005.

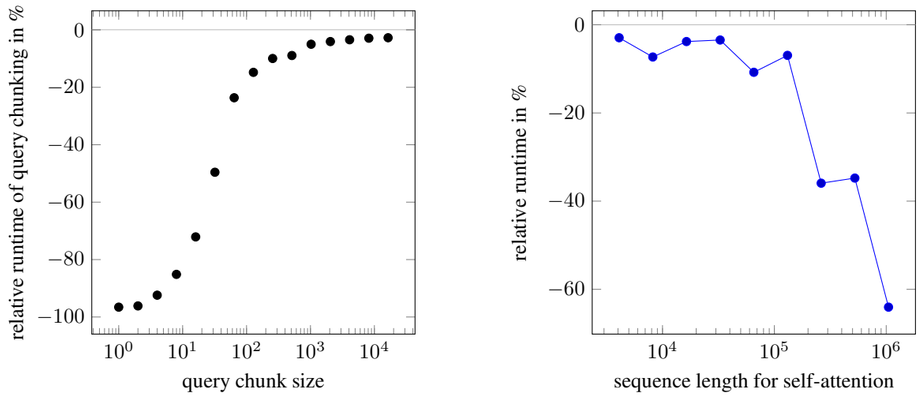


Figure 5: **Left:** Relative runtime of self-attention on sequence length 2^{15} using query chunking compared to standard attention. **Right:** Relative runtime of self-attention using query chunking compared to our memory-efficient algorithm, where both are restricted to the same amount of memory.

5.4 Comparison to Query Chunking

The algorithms introduced in this work chunk both the keys and the queries. Chunking the only queries has been explored already by Kitaev et al. (2020), but it is folklore that it slows down the computation significantly. In Figure 5 (left), we plot the runtime of self-attention using query-chunking for different query chunk sizes compared to dense self-attention: we see that for small chunk sizes (e.g. ≤ 64) the performance suffers indeed, but for large chunk sizes, the loss of performance is less significant. So, while lower memory consumption can be achieved by query chunking alone, small values for query chunking are impractical.

In comparison to query chunking, memory-efficient attention can save additional memory by chunking also the keys. This can help to keep the query chunk size at a desirable point given a fixed memory limit. In Figure 5, we constrained query chunking to the amount of memory that is used by memory-efficient attention with the default settings for key and query chunk size (see Table 2, "Memory overhead of memory-efficient att.", we rounded the query chunk size towards the benefit of query chunking). We see that as the sequence length increases, query chunking eventually slows down significantly as the query chunk size has to be lowered to ≤ 64, while memory-efficient attention does not suffer a major slowdown (see Table 2, "Relative compute speed"). So, in memory-constrained scenarios, memory-efficient attention can outperform query chunking.

6 Related Work

After publishing our initial draft, we were made aware that Jang et al. (2019) already observed that the division of the softmax operation can be delayed until the end of the attention operation ("lazy softmax"), similar to our Equation (1). But their paper does not discuss memory complexity at all. They also do not address numerical stability or backpropagation, and, as far as we know, there is no publicly available implementation of their work. Instead they use this algorithm to reduce the memory *bandwidth* for inference when sharding key-value pairs across multiple chips.

Dao et al. (2022) provide a CUDA implementation of memory-efficient attention and demonstrate that the reduced memory requirements can translate to significant speedups on GPUs. One reason why we do not observe the same performance gains in this paper is that standard self-attention already balances the available FLOPs and memory bandwidth of TPUs.

7 Conclusion

This paper presents a simple trick to reduce the memory requirement of (self-)attention dramatically, which appears to have been simply overlooked by the community. We hope that this short paper raises awareness of the fact that attention is not intrinsically memory-hungry, which may allow us to revisit some of the design choices in popular neural architectures and hardware architectures.

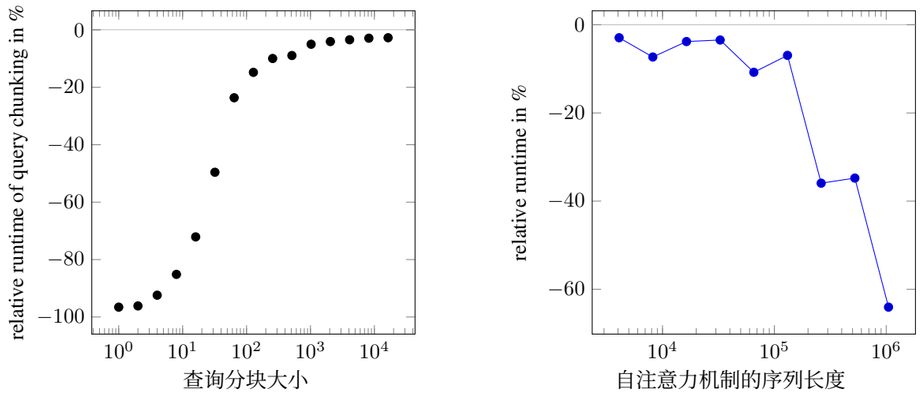


图5：左侧为在序列长度为 2^{15} 的情况下，采用查询分块技术的自注意力机制与标准注意力机制的相对运行时间对比；右侧为在相同内存限制条件下，采用查询分块技术的自注意力机制与我们所设计的的高内存效率算法的相对运行时间对比。

5.4 与查询分块技术的比较

本研究所提出的算法会对键和查询都进行分块处理。仅对查询进行分块的处理方式早已被Kitaev等人（2020年）研究过，但有一种普遍的观点认为这会显著降低计算速度。在图5（左侧）中，我们展示了在不同查询分块大小下，采用查询分块机制的自注意力机制与密集型自注意力机制的运行时间对比：可以看出，当分块大小较小时（例如 ≤ 64），性能确实会受到影响；而当分块大小较大时，性能下降的程度则没那么明显。因此，尽管仅通过查询分块就能实现更低的内存消耗，但过小的分块尺寸在实际应用中并不实用。

与查询分块相比，高内存效率注意力机制还能通过对键值进行分块来进一步节省内存。在内存限制固定的情况下，这有助于将查询分块大小维持在理想水平。如图5所示，我们将查询分块所使用的内存量限制在了高内存效率注意力机制在键值及查询分块大小默认设置下的内存消耗范围内（参见表2“高内存效率注意力机制的内存开销”一栏，其中查询分块大小是按照有利于查询分块的方式取整的）。我们可以看到，随着序列长度的增加，由于必须将查询分块大小降低到 ≤ 64，查询分块方式最终会显著变慢，而高内存效率注意力机制则不会出现大幅性能下降（参见表2“相对计算速度”一栏）。因此，在内存受限的场景中，高内存效率注意力机制的表现往往优于查询分块方式。

6 相关工作

在发布初步稿件后，我们得知Jang等人（2019）早已发现，与我们的公式（1）类似，softmax运算的分步处理可以推迟到注意力运算结束时进行，即采用“懒惰softmax算法”。不过他们的论文完全没有涉及内存复杂度的问题，也未探讨数值稳定性或反向传播的相关内容；据我们所知，目前也不存在他们该工作的公开实现版本。相反，他们是利用这种算法在将键值对分片处理到多颗芯片上时，降低推理过程所需的内存带宽。Dao等人（2022年）提出了基于CUDA框架的高内存效率注意力机制实现方案，并证明较低的内存需求能够显著提升在图形处理器上的运行速度。本文未能观察到同样的性能提升，其原因之一在于传统的自注意力机制本身就已经能够合理利用张量处理单元的可用浮点运算次数与内存带宽。

7 结论

本文介绍了一种简单的方法，能够大幅降低（自）注意力机制的内存需求，而这一方法似乎一直被学术界忽视。我们希望这篇简短的论文能够让大家意识到，注意力机制本身并非必然会消耗大量内存，这或许有助于我们重新审视各类主流神经网络架构及硬件架构中的某些设计选择。

Acknowledgements

We want to thank Andrew Jaegle for discussions on this paper, and for experimenting with memory-efficient attention in the context of Perceiver (Hawthorne et al., 2022). We are glad to see that the algorithm proposed here has already found interest and would like to thank Rezaei (2021) and Wang (2022) for reimplementations in JAX and PyTorch with additional features like masking. We also want to thank DeLesley Hutchins for detailed feedback on our draft.

References

Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In Yoshua Bengio and Yann LeCun (eds.), *ICLR*, 2015.

James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL `http://github.com/google/jax`.

Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *CoRR*, abs/1604.06174, 2016.

Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *CoRR*, abs/1904.10509, 2019. URL `http://arxiv.org/abs/1904.10509`.

Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamás Sarlós, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy J. Colwell, and Adrian Weller. Rethinking attention with performers. *CoRR*, abs/2009.14794, 2020. URL `https://arxiv.org/abs/2009.14794`.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness, 2022. URL `https://arxiv.org/abs/2205.14135`.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. In Jill Burstein, Christy Doran, and Thamar Solorio (eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, pp. 4171–4186. Association for Computational Linguistics, 2019. doi: 10.18653/v1/n19-1423. URL `https://doi.org/10.18653/v1/n19-1423`.

Curtis Hawthorne, Andrew Jaegle, Cătălina Cangea, Sebastian Borgeaud, Charlie Nash, Mateusz Malinowski, Sander Dieleman, Oriol Vinyals, Matthew Botvinick, Ian Simon, Hannah Sheahan, Neil Zeghidour, Jean-Baptiste Alayrac, João Carreira, and Jesse Engel. General-purpose, long-context autoregressive modeling with perceiver AR, 2022. URL `https://arxiv.org/abs/2202.07765`.

Jonathan Heek, Anselm Levskaya, Avital Oliver, Marvin Ritter, Bertrand Rondepierre, Andreas Steiner, and Marc van Zee. Flax: A neural network library and ecosystem for JAX, 2020. URL `http://github.com/google/flax`.

Hanhwi Jang, Joonsung Kim, Jae-Eon Jo, Jaewon Lee, and Jangwoo Kim. Mnnfast: A fast and scalable system architecture for memory-augmented neural networks. In *2019 ACM/IEEE 46th Annual International Symposium on Computer Architecture (ISCA)*, pp. 250–263, 2019.

Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451*, 2020.

Xuezhe Ma, Xiang Kong, Sinong Wang, Chunting Zhou, Jonathan May, Hao Ma, and Luke Zettlemoyer. Luna: Linear unified nested attention. *CoRR*, abs/2106.01540, 2021. URL `https://arxiv.org/abs/2106.01540`.

Jiezhong Qiu, Hao Ma, Omer Levy, Wen-tau Yih, Sinong Wang, and Jie Tang. Blockwise self-attention for long document understanding. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 2555–2565, 2020.

Hongyu Ren, Hanjun Dai, Zihang Dai, Mengjiao Yang, Jure Leskovec, Dale Schuurmans, and Bo Dai. Combiner: Full attention transformer with sparse computation cost. *arXiv preprint arXiv:2107.05768*, 2021.

Amin Rezaei. Memory efficient attention, 2021. URL `https://github.com/AminRezaei0x443/memory-efficient-attention`.

Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Trans. Assoc. Comput. Linguistics*, 9:53–68, 2021. URL `https://transacl.org/ojs/index.php/tacl/article/view/2405`.

Zhuoran Shen, Mingyuan Zhang, Haiyu Zhao, Shuai Yi, and Hongsheng Li. Efficient attention: Attention with linear complexities. In *IEEE Winter Conference on Applications of Computer Vision, WACV 2021, Waikoloa,*

致谢

我们要感谢Andrew Jaegle就本文展开的讨论，以及他在Perceiver（Hawthorne等人，2022年）项目中对高内存效率注意力机制进行的实验。令人欣慰的是，此处提出的算法已经引起了不少关注；同时我们也要感谢Rezaei（2021年）和Wang（2022年）使用JAX和PyTorch重新实现了该算法，并加入了掩码等功能。此外，我们还要感谢DeLesley Hutchins对我们初稿给出的详尽反馈。

参考文献

Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In Yoshua Bengio and Yann LeCun (eds.), *ICLR*, 2015.

James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL `http://github.com/google/jax`.

Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *CoRR*, abs/1604.06174, 2016.

Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *CoRR*, abs/1904.10509, 2019. URL `http://arxiv.org/abs/1904.10509`.

Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamás Sarlós, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy J. Colwell, and Adrian Weller. Rethinking attention with performers. *CoRR*, abs/2009.14794, 2020. URL `https://arxiv.org/abs/2009.14794`.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness, 2022. URL `https://arxiv.org/abs/2205.14135`.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. In Jill Burstein, Christy Doran, and Thamar Solorio (eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, pp. 4171–4186. Association for Computational Linguistics, 2019. doi: 10.18653/v1/n19-1423. URL `https://doi.org/10.18653/v1/n19-1423`.

Curtis Hawthorne, Andrew Jaegle, Cătălina Cangea, Sebastian Borgeaud, Charlie Nash, Mateusz Malinowski, Sander Dieleman, Oriol Vinyals, Matthew Botvinick, Ian Simon, Hannah Sheahan, Neil Zeghidour, Jean-Baptiste Alayrac, João Carreira, and Jesse Engel. General-purpose, long-context autoregressive modeling with perceiver AR, 2022. URL `https://arxiv.org/abs/2202.07765`.

Jonathan Heek, Anselm Levskaya, Avital Oliver, Marvin Ritter, Bertrand Rondepierre, Andreas Steiner, and Marc van Zee. Flax: A neural network library and ecosystem for JAX, 2020. URL `http://github.com/google/flax`.

Hanhwi Jang, Joonsung Kim, Jae-Eon Jo, Jaewon Lee, and Jangwoo Kim. Mnnfast: A fast and scalable system architecture for memory-augmented neural networks. In *2019 ACM/IEEE 46th Annual International Symposium on Computer Architecture (ISCA)*, pp. 250–263, 2019.

Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451*, 2020.

Xuezhe Ma, Xiang Kong, Sinong Wang, Chunting Zhou, Jonathan May, Hao Ma, and Luke Zettlemoyer. Luna: Linear unified nested attention. *CoRR*, abs/2106.01540, 2021. URL `https://arxiv.org/abs/2106.01540`.

Jiezhong Qiu, Hao Ma, Omer Levy, Wen-tau Yih, Sinong Wang, and Jie Tang. Blockwise self-attention for long document understanding. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 2555–2565, 2020.

Hongyu Ren, Hanjun Dai, Zihang Dai, Mengjiao Yang, Jure Leskovec, Dale Schuurmans, and Bo Dai. Combiner: Full attention transformer with sparse computation cost. *arXiv preprint arXiv:2107.05768*, 2021.

Amin Rezaei. Memory efficient attention, 2021. URL `https://github.com/AminRezaei0x443/memory-efficien`t-注意力机制。奥尔科·罗伊、穆罕默德·萨法尔、阿希什·瓦斯瓦尼与大卫·格兰吉耶。基于路由变换器的高效内容驱动稀疏注意力机制。《计算机语言学协会会刊》，9:53–68，2021年。网址：https://transacl.org/ojs/index.php/tacl/article/view/2405。沈卓然、张明远、赵海宇、易帅与李洪生。高效注意力机制：具有线性时间复杂度的注意力算法。载于2021年IEEE计算机视觉应用冬季会议——WACV 2021，怀科洛阿。

Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Trans. Assoc. Comput. Linguistics*, 9:53–68, 2021. URL `https://transacl.org/ojs/index.php/tacl/article/view/2405`.

Zhuoran Shen, Mingyuan Zhang, Haiyu Zhao, Shuai Yi, and Hongsheng Li. Efficient attention: Attention with linear complexities. In *IEEE Winter Conference on Applications of Computer Vision, WACV 2021, Waikoloa,*

HI, USA, January 3-8, 2021, pp. 3530–3538. IEEE, 2021. doi: 10.1109/WACV48630.2021.00357. URL https://doi.org/10.1109/WACV48630.2021.00357.

Yi Tay, Mostafa Dehghani, Samira Abnar, Yikang Shen, Dara Bahri, Philip Pham, Jinfeng Rao, Liu Yang, Sebastian Ruder, and Donald Metzler. Long range arena : A benchmark for efficient transformers. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. URL https://openreview.net/forum?id=qVyeW-grC2k.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pp. 5998–6008, 2017.

Phil Wang. Memory efficient attention Pytorch, 2022. URL https://github.com/lucidrains/memory-efficient-attention-p

Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.

Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontañón, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin (eds.), *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020. URL https://proceedings.neurips.cc/paper/2020/hash/c8512d142a2d849725f31a9a7a361ab9-Abstract.html.

HI, USA, January 3-8, 2021, pp. 3530–3538. IEEE, 2021. doi: 10.1109/WACV48630.2021.00357. URL https://doi.org/10.1109/WACV48630.2021.00357.

Yi Tay, Mostafa Dehghani, Samira Abnar, Yikang Shen, Dara Bahri, Philip Pham, Jinfeng Rao, Liu Yang, Sebastian Ruder, and Donald Metzler. Long range arena : A benchmark for efficient transformers. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. URL https://openreview.net/forum?id=qVyeW-grC2k.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pp. 5998–6008, 2017.

Phil Wang. Memory efficient attention Pytorch, 2022. URL https://github.com/lucidrains/memory-efficient-attention-p

Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.

Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontañón, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin (eds.), *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020. URL https://proceedings.neurips.cc/paper/2020/hash/c8512d142a2d849725f31a9a7a361ab9-Abstract.html.